05010105DS15 - Full Stack Web Development-II (P)

Node.js & Express.js Practical Lab Manual

Table of Contents

Prerequisites
Lab 1 & 10: Asynchronous Console Input
Lab 2: CRUD Operations on a Text File
Lab 3: Basic Express Server
Lab 4: Express GET and POST Requests
Lab 5: RESTful Routes for User Management
Lab 6: Real-time Chat using WebSocket
Lab 7: Node.js + MongoDB Native CRUD
Lab 8 & 9: RESTful API with Express & MongoDB
Lab 11 & 12: User Auth (JWT, Registration, Login)
Lab 13: Dockerize Node.js App
Lab 14: REST API for a Blog
Lab 15: Passport.js Authentication

Before starting, ensure you have **Node.js** and **MongoDB** installed on your system. Create a new folder for your labs and initialize a Node.js project:

```
mkdir node-labs
cd node-labs

npm init -y
```

Lab 1 & 10: Asynchronous Console Input

Question 1 & 10: Write a Node.js program that reads a user's name from the console and prints a greeting message asynchronously. *(Note: Questions 1 and 10 are identical).*

Code (lab1.js):

```
const readline = require('readline');
```

```
const rl = readline.createInterface({
   input: process.stdin,
   output: process.stdout
});

console.log("Program started...");

rl.question('Please enter your name: ', (name) => {
   // Simulating an asynchronous operation with setTimeout
   setTimeout(() => {
              console.log(`\nHello, ${name}! Welcome to Node.js asynchronously.`);
       rl.close();
   }, 1000);

});
```

Execution & Output:

```
node lab1.js
# Output:
# Program started...
# Please enter your name: John
# [Waits 1 second...]
```

Hello, John! Welcome to Node.js asynchronously.

Lab 2: CRUD Operations on a Text File

Question 2: Write a Node.js script to create, read, update, and delete (CRUD) a text file using the fs module.

Code (lab2.js):

```
const fs = require('fs').promises;
const fileName = 'data.txt';

async function fileCRUD() {
  try {
    // 1. Create (Write)
    await fs.writeFile(fileName, 'Hello, this is initial text.\n');
    console.log('File Created!');

    // 2. Read
    let data = await fs.readFile(fileName, 'utf8');
    console.log('File Content after Create:\n' + data);
```

```
        // 3. Update (Append)
        await fs.appendFile(fileName, 'This is appended text.');
        console.log('File Updated!');

        // Read again to show update
        data = await fs.readFile(fileName, 'utf8');
        console.log('File Content after Update:\n' + data);

        // 4. Delete
        await fs.unlink(fileName);
        console.log('File Deleted successfully!');
    } catch (err) {
        console.error('Error:', err);
    }
}

fileCRUD();
```

Execution & Output:

```
node lab2.js
# Output:
```

```
# File Created!
# File Content after Create:
# Hello, this is initial text.
#
# File Updated!
# File Content after Update:
# Hello, this is initial text.
# This is appended text.

# File Deleted successfully!
```

Lab 3: Basic Express Server

Question 3: Install Express.js and set up a basic Express server.

Setup:

```
npm install express
```

Code (lab3.js):

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
   res.send('<h1>Hello, Express.js Server is running!</h1>');
});

app.listen(PORT, () => {
   console.log(`Server is running on http://localhost:${PORT}`);

});
```

Execution & Output:

```
node lab3.js
# Output:

# Server is running on http://localhost:3000
```

Lab 4: Express GET and POST Requests

Question 4: Create an Express.js application with routes for handling GET and POST requests for user data.

Code (lab4.js):

```
const express = require('express');
const app = express();
app.use(express.json()); // Middleware to parse JSON bodies

app.get('/api/user', (req, res) => {
    res.json({ message: "Send a POST request to this endpoint with user data." });
});

app.post('/api/user', (req, res) => {
  const userData = req.body;
  res.status(201).json({
      message: 'User data received successfully!',
      data: userData
  });
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

Execution & Output:

```
node lab4.js
# In another terminal:
curl -X POST http://localhost:3000/api/user -H "Content-Type: application/json" -d '{"name":"Alice", "age":25}'

# Output:

# {"message":"User data received successfully!","data":{"name":"Alice","age":25}}
```

Lab 5: RESTful Routes for User Management

Question 5: Create RESTful routes (GET, POST, PUT, DELETE) for a user management system.

Code (lab5.js):

```
const express = require('express');
const app = express();
app.use(express.json());

let users = [{ id: 1, name: 'John Doe' }];

// GET all
app.get('/users', (req, res) => res.json(users));

// POST create
app.post('/users', (req, res) => {
  const newUser = { id: Date.now(), name: req.body.name };
  users.push(newUser);
  res.status(201).json(newUser);
});

// PUT update
app.put('/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).send('User not found');
  user.name = req.body.name;
  res.json(user);
});
```

```
// DELETE
app.delete('/users/:id', (req, res) => {
  users = users.filter(u => u.id !== parseInt(req.params.id));
  res.json({ message: 'User deleted' });
});


app.listen(3000, () => console.log('REST API running on port 3000'));
```

Lab 6: Real-time Chat using WebSocket

Question 6: Create a real-time chat application using WebSocket and Socket.io.

Setup:

```
npm install socket.io
```

Code (lab6.js - Server):

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

app.get('/', (req, res) => {
  res.sendFile(__dirname + '/index.html');
});

io.on('connection', (socket) => {
  console.log('A user connected');
  socket.on('chat message', (msg) => {
    io.emit('chat message', msg); // Broadcast to everyone
  });
            socket.on('disconnect', () => console.log('User
disconnected'));
});
```

```
server.listen(3000, () => console.log('Chat server running on port 3000'));
```

Code (index.html - Client):

```
<!DOCTYPE html>
<html>
<body>
  <ul id="messages"></ul>
  <form id="form" action="">
                    <input id="input" autocomplete="off" /><button>Send</button>
  </form>
  <script src="/socket.io/socket.io.js"></script>
  <script>
    var socket = io();
    var form = document.getElementById('form');
    var input = document.getElementById('input');

    form.addEventListener('submit', function(e) {
      e.preventDefault();
      if (input.value) {
        socket.emit('chat message', input.value);
        input.value = '';
```

```
        }
    });

    socket.on('chat message', function(msg) {
        var item = document.createElement('li');
        item.textContent = msg;

document.getElementById('messages').appendChild(item);
    });
  </script>
</body>

</html>
```

Lab 7: Node.js + MongoDB Native CRUD

Question 7: Create a Node.js application that connects to MongoDB and performs CRUD operations (Create, Read, Update, Delete) on a "users" collection.

Setup:

```
npm install mongodb
```

Code (lab7.js):

```
const { MongoClient } = require('mongodb');
const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const db = client.db('labDB');
    const collection = db.collection('users');

    // Create
        const insertResult = await collection.insertOne({ name: "Alice", role: "Admin" });
    console.log("Inserted:", insertResult.insertedId);

    // Read
    const user = await collection.findOne({ name: "Alice" });
    console.log("Found:", user);

    // Update
```

```
        await collection.updateOne({ name: "Alice" }, { $set: { role: "SuperAdmin" } });
        console.log("Updated Alice's role.");

        // Delete
        await collection.deleteOne({ name: "Alice" });
        console.log("Deleted Alice.");

    } finally {
        await client.close();
    }
}


run();
```

Lab 8 & 9: RESTful API with Express & MongoDB

Question 8 & 9: Develop an Express API that connects to MongoDB and performs CRUD operations. Design a RESTful API using Express.js and MongoDB.

Setup:

```
npm install mongoose
```

Code (lab8_9.js):

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

mongoose.connect('mongodb://localhost:27017/restDB')
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.log(err));

// Schema & Model
const UserSchema = new mongoose.Schema({ name: String, age: Number });
const User = mongoose.model('User', UserSchema);

// GET All
app.get('/api/users', async (req, res) => {
  const users = await User.find();
```

```
  res.json(users);
});

// POST Create
app.post('/api/users', async (req, res) => {
  const newUser = new User(req.body);
  await newUser.save();
  res.status(201).json(newUser);
});

// PUT Update
app.put('/api/users/:id', async (req, res) => {
                              const updated = await
User.findByIdAndUpdate(req.params.id, req.body, { new: true
});
  res.json(updated);
});

// DELETE
app.delete('/api/users/:id', async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.json({ message: 'Deleted Successfully' });
});
```

```
app.listen(3000, () => console.log('API running on port 3000'));
```

Lab 11 & 12: User Auth (JWT, Registration, Login)

Question 11 & 12: Implement user authentication using JSON Web Token (JWT) in an Express.js application. Develop a RESTful API using Express.js and MongoDB that allows users to register and log in.

Setup:

```
npm install jsonwebtoken bcryptjs
```

Code (lab11_12.js):

```
const express = require('express');
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
```

```
const app = express();
app.use(express.json());

mongoose.connect('mongodb://localhost:27017/authDB');

const User = mongoose.model('AuthUser', new mongoose.Schema({
  username: String,
  password: String
}));

const SECRET = 'my_super_secret_key';

// Register
app.post('/register', async (req, res) => {
                    const hashedPassword = await bcrypt.hash(req.body.password, 10);
      const user = new User({ username: req.body.username, password: hashedPassword });
  await user.save();
  res.status(201).send('User registered');
});

// Login
```

```
app.post('/login', async (req, res) => {
        const user = await User.findOne({ username: req.body.username });
  if (!user) return res.status(400).send('User not found');

   const validPass = await bcrypt.compare(req.body.password, user.password);
  if (!validPass) return res.status(400).send('Invalid password');

  const token = jwt.sign({ _id: user._id }, SECRET);
  res.header('auth-token', token).json({ token });
});

// Protected Route Middleware
const verifyToken = (req, res, next) => {
  const token = req.header('auth-token');
  if (!token) return res.status(401).send('Access Denied');
  try {
    req.user = jwt.verify(token, SECRET);
    next();
  } catch (err) {
    res.status(400).send('Invalid Token');
  }
};
```

```
app.get('/profile', verifyToken, (req, res) => {
   res.send('Welcome to your protected profile!');
});


app.listen(3000, () => console.log('Auth API on port 3000'));
```

Lab 13: Dockerize Node.js App

Question 13: Dockerize a Node.js application.

File 1: **package.json**

File 2: **Dockerfile**

```
# Use official Node runtime
FROM node:18-alpine

# Set working directory
WORKDIR /app
```

```
# Copy package.json and install dependencies
COPY package*.json ./
RUN npm install

# Copy rest of the application code
COPY . .

# Expose port
EXPOSE 3000

# Start command

CMD ["npm", "start"]
```

File 3: .dockerignore

```
node_modules

.git
```

Lab 14: REST API for a Blog

Question 14: Develop a REST API for a blog application using Node.js and Express.js.

Code (lab14.js):

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

mongoose.connect('mongodb://localhost:27017/blogDB');

const PostSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: String,
  createdAt: { type: Date, default: Date.now }
});

const Post = mongoose.model('Post', PostSchema);
```

```
app.post('/posts', async (req, res) => {
  const post = new Post(req.body);
  await post.save();
  res.status(201).json(post);
});

app.get('/posts', async (req, res) => {
  res.json(await Post.find());
});

app.delete('/posts/:id', async (req, res) => {
  await Post.findByIdAndDelete(req.params.id);
  res.send('Post deleted');
});


app.listen(3000, () => console.log('Blog API running on port 3000'));
```

Lab 15: Passport.js Authentication

Question 15: Implement user authentication using Passport.js (Google or Local Strategy).

Setup:

```
npm install passport passport-local express-session
```

Code (lab15.js):

```
const express = require('express');
const session = require('express-session');
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;

const app = express();
app.use(express.urlencoded({ extended: false }));

// Dummy DB
const users = [{ id: 1, username: 'admin', password: 'password123' }];

// Session setup
```

```
app.use(session({ secret: 'secret', resave: false, saveUninitialized: false }));
app.use(passport.initialize());
app.use(passport.session());

// Passport configuration
passport.use(new LocalStrategy((username, password, done) => {
  const user = users.find(u => u.username === username);
      if (!user) return done(null, false, { message: 'Incorrect username.' });
     if (user.password !== password) return done(null, false, { message: 'Incorrect password.' });
  return done(null, user);
}));

passport.serializeUser((user, done) => done(null, user.id));
passport.deserializeUser((id, done) => {
  const user = users.find(u => u.id === id);
  done(null, user);
});

// Routes
app.post('/login', passport.authenticate('local', {
  successRedirect: '/dashboard',
```

```
  failureRedirect: '/login-failed'
}));

app.get('/dashboard', (req, res) => {
  if (req.isAuthenticated()) {
            res.send(`Hello ${req.user.username}, welcome to dashboard!`);
  } else {
    res.status(401).send('Please log in first.');
  }
});

app.get('/login-failed', (req, res) => res.send('Login Failed!'));


app.listen(3000, () => console.log('Passport App running on 3000'));
```